1.当时为什么决定选用RAG这个技术框架

• 选 RAG 主要是一个工程取舍：在这个项目里它比“纯大模型直答”或“微调优先”更合适。

1. 业务上要“基于私有知识库回答”
   项目核心是 Agent + 知识库问答。我们需要让模型回答绑定到用户上传的文档，而不是只靠参数记忆。RAG 能把外部知识在推理时
   注入，降低幻觉。
2. 可持续迭代，成本低
   这个项目支持文档上传后即时生效（Markdown 解析分块、向量化、入库），不需要每次改知识都重新训练模型。对中小团队来说，
   维护和迭代成本明显更低。
3. 和现有架构契合
   我们后端已经是 Spring Boot + PostgreSQL，所以直接用 pgvector 做相似度检索，结构化数据和向量数据统一在一套存储里，部
   署和运维都更简单。
   实际链路是：DocumentFacadeServiceImpl 做分块+embedding，ChunkBgeM3Mapper 用 <-> 检索，Agent 通过 KnowledgeTools 调
   RagService 把召回片段注入到 Think-Execute 流程里。

一句话总结：这个项目选 RAG，是为了在可控成本下，把“回答准确性、知识实时性、工程可落地性”同时拿到。

2.你最终选择的是哪个RAG/哪个向量库？ 为什么选他？

• 我最终选的是：

- RAG方案：自建 RAG 流水线（文档分块 -> bge-m3 向量化 -> 召回 TopK -> 注入 Agent 上下文）
- 向量库：PostgreSQL + pgvector

为什么这么选：

1. 工程复杂度最低
   项目本身就用 PostgreSQL，直接上 pgvector，不用再引入 Milvus/ES 等新基础设施。
2. 数据一致性更好
   结构化数据（知识库、文档）和向量数据在同库，事务、权限、备份策略统一，运维简单。
3. 检索链路够用且可优化
   当前通过 embedding <-> query_vector 做相似度检索，并按 kb_id 过滤；中等规模下性能和

准确性都满足项目目标。
（我在设计上也预留了 ivfflat 这类索引优化路径。）

4. 与 Agent 集成顺滑
在项目里由 KnowledgeTools -> RagService -> pgvector 直接走工具调用，能自然接入 Think-Execute 循环。

如果后续数据量上到千万级、并发更高，我会再评估迁移到专门向量引擎。

3.为什么不用全文索引 类似于百度或谷歌那种的关键词索引 而是使用RAG向量检索的方式

• 因为我们的问题类型是“语义问答”，不是“关键词检索”，所以主链路选了 RAG 向量检索。

1. 关键词检索对“同义表达”不鲁棒
用户问“怎么做连接复用”，文档写“连接池复用机制”，倒排索引容易漏召回；向量检索更容易把语义接近的片段找出来。
2. 这个项目要给 LLM 喂上下文，不只是返回链接
我们要把最相关片段直接注入 Agent 推理链路（KnowledgeTool -> RagService -> TopK chunks），向量召回更适合这一步。
3. 文档是非结构化内容（Markdown段落）
对这类文本，按 chunk 做 embedding + 相似度检索，通常比纯 BM25 在问答场景下效果更稳。
4. 工程取舍
我们已经用 PostgreSQL，pgvector 接入成本低，能快速落地。

另外不是“全文索引没价值”。
更好的做法其实是混合检索（BM25 + 向量 + 重排）。当前项目先用向量主链路，是为了先把可用性做出来，后续可再加关键词召回做
补充。

4.你刚刚一直强调“幻觉”这个概念，那为什么不用更权威的全文检索来避免模型胡编呢

• “权威全文检索”本身不能直接消除幻觉，它只能提升“找文档”的可靠性，不能保证“模型生成”不胡编。

在我们项目里，避免幻觉是三层：

1. 先把相关上下文喂给模型
我们用 RAG 向量检索召回语义最相关的 chunk，减少“无依据生成”。
2. 再做回答约束
系统提示里会要求优先基于知识库内容回答，缺上下文先检索，不让模型自由发挥。

3. 工程上可继续加强
   你说的全文检索其实应该作为补充而不是替代。最佳实践通常是“混合检索”：BM25/全文 + 向量 + 重排，再加“无证据就拒答/澄
   清”和引用片段展示。

所以核心点是：
不是“全文检索 vs RAG”二选一。全文检索解决词项匹配和可解释性，RAG向量解决语义召回；两者结合，才更能压低幻觉。

5.如果用开源全文检索在本地部署、自己喂私有数据，也能够实现从知识库中搜索用户相关的问题，你为什么没采取这种方式呢

• 因为当时目标是先把 Agent 主链路快速做成“可用产品”，在这个约束下我选了 PostgreSQL + pgvector，没先上独立全文检索引擎。

核心原因就三点：

1. 交付优先级
   我们当时同时在做 Think-Execute、工具系统、SSE 实时流。再引入 ES/OpenSearch 会显著增加部署、运维和数据同步复杂度，首
   版节奏会被拖慢。
2. 场景匹配
   这个项目主要是问答式语义检索（自然语言问法和文档措辞不一致很常见），向量检索在首版里收益更直接。
3. 现有技术栈复用
   业务数据本来就在 PostgreSQL，向量和结构化数据同库管理，事务与权限策略统一，工程成本最低。

但你的方案是对的。
如果项目进入下一阶段，我会加“开源全文检索 + 向量”的混合检索，而不是只保留单路向量。这样能同时拿到关键词精确匹配和语义
召回能力，进一步压低幻觉和漏召回。

6.你选用的向量维度是多少维

• 当前实现里代码没有硬编码维度，是直接接 bge-m3 返回的向量长度。
按 bge-m3 的输出，实际用的是 1024 维。

7.这个维度为什么重要

• 很重要，主要影响三件事：

1. 检索效果上限
   维度太低，语义信息压缩过度，容易召回不准；维度合理，语义区分能力更好。
2. 性能和成本
   维度越高，向量存储更大、索引更重、相似度计算更慢，延迟和资源占用都会上去。
3. 系统一致性
   模型输出维度必须和库里 vector(n) 一致，不一致会直接写入/查询失败。

一句话：维度是在“准确率”和“延迟/成本”之间做平衡的核心参数。

8.你的切分策略是什么?

• 当前项目的切分策略是：

- 按 Markdown 标题（Heading） 切分
- 每个 chunk = “一个标题 + 到下一个标题前的全部内容”（不区分 h1/h2/h3，遇到任意下一个标题就截断）
- 表格会尽量保留原始 Markdown 格式
- 入库时每个 section 生成一个 chunk，检索时按向量相似度取 TopK=3

另外有个实现细节：现在 embedding 用的是 section 的 title（标题）来向量化，不是 title+content。这会更轻，但对内容长段落的语义覆盖会弱一些。

9.那按照你这种分段的chunk切法 假设有一个段落只有一百个字 有一个段落有一万字 那怎么办

• 这个问题你抓得很准。按我当前这版“按标题切”，确实会出现你说的长度极不均衡问题，这是短板。

当前做法下会有两个风险：

1. 超长 chunk（1万字）语义被稀释
   召回时相关片段不够精准，模型读取成本也高。
2. 超短 chunk（100字）上下文不足
   信息密度不够，回答容易缺细节。

更合理的改法是两级切分：

1. 先结构切分：按标题分 section
2. 再长度切分：对每个 section 按 token 窗口二次切块（比如 600-800 tokens，10%-20% overlap）
3. 查询时召回多个子块，再按文档/标题聚合去重后喂给模型

如果你面试追问“为什么现在没做”，我会直接承认：首版优先打通端到端链路，长度归一化切分是下一步必须补的检索质量项。

10.你的chunksize是多少 为什么这样设置 是拍脑袋还是有依据的

• 当前这版其实没有显式 chunkSize 参数。
切分规则是“按 Markdown 标题切一段到下一标题”，所以 chunk 大小是随文档结构自然变化的，不是固定 500/800 token 那种配置。

所以你问“是不是拍脑袋”，我会这么回答：

1. 现状上
   是偏工程快速落地的策略，不是严格评测驱动的最优参数。
2. 依据上
   有结构依据（标题语义边界），但没有做系统性的长度参数实验（因为还没做二次 token 切分）。
3. 改进方向
   下一步会改成“结构切分 + token 二次切分”，并通过离线评测定 chunkSize（例如 600-800 tokens）和 overlap，而不是凭经验
   定值。

11. 你怎么评价你的RAG效果的好坏

• 我会从这 4 个维度评估，不看单一指标：

1. 检索质量
   看 Recall@K、MRR、NDCG，本质是“该召回的有没有被召回、排序对不对”。
2. 答案质量
   看“是否基于已召回证据回答、是否胡编、是否答非所问”。可用人工标注先做，再逐步半自动化。
3. 端到端体验
   看首包时延、总时延、稳定性。RAG再准但响应慢，产品体验也不合格。
4. 线上业务结果
   看用户追问率、重试率、人工兜底率、满意度。

如果只评价我这个项目当前版本：
“可用，但不够工程化”。有基础链路和初步效果，但缺少标准化评测集和自动回归，这块是明显短板。